BASES DE DATOS

Implementación física de la Red Social Estudiantil Pascualina

PostgreSQL · MySQL · SQLITE

Institución universitaria pascual bravo

Bases de Datos

Jesus David Vargas Henao

Jose Daniel Vargas Henao

Oralia Cortes Grajales

24 octubre de 2026

1. Introducción

Pascualina es una propuesta de red social para fortalecer la comunicación entre estudiantes, facilitar el intercambio de conocimientos, permitir el seguimiento entre compañeros y apoyar la creación de grupos, publicaciones y eventos. Esta entrega retoma el modelo relacional de 15 tablas desarrollado previamente y lo adapta a tres sistemas gestores de bases de datos relacionales (SGBDR). Además, incorpora un atributo JSON para demostrar el tratamiento de información semiestructurada.

2. Objetivos

Objetivo general. Diseñar y documentar la implementación física de Pascualina en PostgreSQL, MySQL y SQLite, conservando las relaciones esenciales y justificando las adaptaciones técnicas de cada motor.

Objetivos específicos. (1) Actualizar el diccionario genérico y elaborar tres diccionarios físicos; (2) desarrollar un script DDL por motor; (3) incorporar datos semiestructurados mediante JSON; (4) comparar tipos de datos, claves, restricciones y reglas de integridad; (5) registrar pruebas reales y presentar conclusiones individuales.

3. ¿Por qué elegimos estos tres motores?

PostgreSQL. Lo elegimos porque ya trabajamos con él en la actividad anterior, lo cual nos permite aprovechar el modelo que construimos y concentrarnos en las diferencias de implementación. Es un gestor relacional de código abierto que ofrece restricciones de integridad y el tipo **JSONB**, útil para consultar información semiestructurada. También nos sirve como punto de referencia para comparar los otros dos motores.

MySQL. Lo seleccionamos por su uso frecuente en aplicaciones web y por su integración con proyectos que necesitan almacenar información relacional de usuarios, publicaciones y grupos. Su tipo **JSON** nos permite comparar cómo se validan y almacenan documentos frente a PostgreSQL. Además, la sintaxis **AUTO_INCREMENT** evidencia diferencias concretas en la creación de identificadores.

SQLITE: porque nos permitió implementar nuestro modelo relacional de una manera más sencilla, sin necesidad de configurar un servidor independiente. Además, consideramos que era una buena alternativa para realizar las pruebas y comparaciones que nos solicita la actividad.

Con SQLite logramos implementar las 15 tablas de nuestro proyecto, incluyendo sus claves primarias, claves foráneas y restricciones de integridad. Esto nos permitió conservar la estructura principal de la base de datos y comprobar su funcionamiento en un entorno diferente.

Otra característica que nos pareció importante fue su capacidad para trabajar con información en formato JSON, ya que nuestro proyecto contempla el almacenamiento de preferencias y otros datos de los estudiantes que pueden variar con el tiempo.

4. Modelo de datos de Pascualina

El modelo conserva las siguientes 15 tablas: **ESTUDIANTE, PERFIL, HABILIDAD, INTERES, ESTUDIANTE_HABILIDAD, ESTUDIANTE_INTERES, SEGUIR, MENTORIA, GRUPO, MIEMBRO_GRUPO, PUBLICACION, COMENTARIO, REACCION, EVENTO y ASISTENTE_EVENTO**. Las tablas de asociación resuelven relaciones muchos a muchos mediante claves foráneas y, cuando corresponde, claves primarias compuestas.

La relación entre ESTUDIANTE y PERFIL debe garantizarse como uno a uno mediante una restricción UNIQUE sobre PERFIL.id_estudiante. Las relaciones de seguimiento y mentoría referencian dos veces la tabla ESTUDIANTE, pero con papeles diferentes: seguidor/seguido y mentor/aprendiz.

5. Adaptaciones físicas entre gestores

Elemento	PostgreSQL	MySQL	SQLite
Identidad	GENERATED AS IDENTITY	AUTO_INCREMENT	INTEGER PRIMARY KEY
Fecha y hora	TIMESTAMP	DATETIME	TEXT con fechas en formato ISO 8601
Texto Unicode	VARCHAR / TEXT	VARCHAR / TEXT (utf8mb4)	TEXT (UTF-8 o UTF-16)
Documento flexible	JSONB	JSON	TEXT + funciones JSON
Validación	CHECK	CHECK (8.0.16+)	CHECK

Las claves primarias nos permiten identificar de manera única cada registro, mientras que las claves foráneas mantienen las relaciones entre las diferentes tablas de nuestra base de datos. Por su parte, las restricciones UNIQUE evitan valores duplicados en determinados campos y las restricciones CHECK nos permiten establecer reglas para validar la información.

Durante la implementación identificamos que, aunque PostgreSQL, MySQL y SQLite admiten estas restricciones, existen diferencias en su funcionamiento. Por ejemplo, en MySQL tuvimos que ajustar algunas restricciones CHECK de las tablas SEGUIR y MENTORIA debido a su incompatibilidad con determinadas acciones referenciales en cascada. En SQLite, las claves foráneas deben habilitarse mediante PRAGMA foreign_keys = ON para garantizar su aplicación en cada conexión.

Estas diferencias nos permitieron comprender que, aunque utilizamos el mismo modelo relacional de 15 tablas, fue necesario realizar adaptaciones específicas para conservar la integridad de los datos en cada gestor.

6. Implementación de datos semiestructurados con JSON

En nuestro proyecto Red Social Estudiantil Pascualina, incorporamos el uso de JSON para almacenar información semiestructurada relacionada con las preferencias de los estudiantes.

Decidimos utilizar este formato porque nos permite guardar información que puede variar entre usuarios sin necesidad de crear nuevas columnas para cada preferencia. Por ejemplo, un estudiante puede configurar sus intereses académicos, las notificaciones que desea recibir o sus preferencias de visualización.

Para implementar esta funcionalidad, incorporamos el campo preferencias_json en la tabla perfil, adaptando su almacenamiento a las características de cada gestor de bases de datos.

PostgreSQL: utilizamos el tipo de dato JSONB, que permite almacenar documentos JSON en un formato binario interno y realizar consultas sobre sus propiedades.

MySQL: utilizamos el tipo de dato JSON, que permite almacenar y consultar documentos JSON. Además, podemos validar que el contenido tenga la estructura de un objeto mediante funciones como JSON_TYPE().

SQLite: utilizamos almacenamiento de texto para representar los documentos JSON, junto con funciones como json_valid() para comprobar que su contenido tenga un formato válido.

7. Proceso de implementación y verificación

Para implementar el modelo físico de nuestra Red Social Estudiantil Pascualina, trabajamos con tres sistemas gestores de bases de datos: PostgreSQL, MySQL y SQLite. Nuestro objetivo fue adaptar el modelo relacional de 15 tablas a las características de cada gestor y comprobar que su estructura se pudiera crear correctamente.

Primero, revisamos el modelo relacional que habíamos desarrollado en la actividad anterior. Identificamos las tablas, sus atributos, claves primarias, claves foráneas y relaciones. A partir de esta estructura, organizamos las instrucciones SQL necesarias para crear la base de datos en cada gestor.

Después, realizamos la implementación en PostgreSQL, definiendo las tablas y sus restricciones de integridad. Este gestor nos sirvió como punto de partida para adaptar posteriormente las instrucciones a los otros motores.

En MySQL, adaptamos la sintaxis SQL y realizamos ajustes en algunas restricciones. Durante las pruebas encontramos errores relacionados con las restricciones CHECK de las tablas SEGUIR y MENTORIA, debido a su combinación con determinadas acciones referenciales en cascada. Corregimos estas definiciones y conseguimos ejecutar el código de creación de las tablas.

Finalmente, implementamos el modelo en SQLite, utilizando los tipos de datos y mecanismos de definición de claves compatibles con este gestor. También tuvimos en cuenta la necesidad de habilitar la comprobación de claves foráneas mediante la instrucción PRAGMA foreign_keys = ON.

Una vez creadas las tablas, ejecutamos consultas para obtener los diccionarios físicos de cada gestor. Estos resultados nos permitieron revisar los nombres de las tablas y columnas, los tipos de datos, las claves y las relaciones definidas.

Como resultado, logramos generar los códigos SQL y los diccionarios físicos correspondientes a PostgreSQL, MySQL y SQLite. Este proceso nos permitió identificar diferencias entre los gestores y comprender la importancia de adaptar las instrucciones SQL según el motor utilizado.

Aunque conseguimos implementar las estructuras y consultar sus metadatos, reconocemos que una verificación más completa requeriría realizar pruebas adicionales de inserción, actualización y eliminación de registros, así como comprobar el comportamiento de todas las restricciones.

8. Dificultades y decisiones técnicas

Durante el desarrollo de la Tarea 4 de nuestro proyecto Red Social Estudiantil Pascualina, encontramos algunas dificultades relacionadas con la adaptación del modelo físico a diferentes gestores de bases de datos. Estas situaciones nos llevaron a investigar los errores, revisar las instrucciones SQL y tomar decisiones técnicas para completar la implementación.

1. Diferencias de sintaxis entre gestores

Una de las primeras dificultades fue comprobar que no podíamos utilizar exactamente el mismo código SQL en PostgreSQL, MySQL y SQLite. Aunque los tres gestores trabajan con bases de datos relacionales, presentan diferencias en la definición de tipos de datos, claves autoincrementales y restricciones.

Para solucionar esto, adaptamos las instrucciones de creación de tablas a la sintaxis de cada gestor, procurando conservar la estructura principal de nuestro modelo relacional.

2. Errores de restricciones en MySQL

Durante la implementación en MySQL encontramos el error 3823, relacionado con las restricciones CHECK de las tablas SEGUIR y MENTORIA. Estas restricciones utilizaban columnas que también participaban en claves foráneas con acciones referenciales en cascada.

Para resolver el problema, retiramos las restricciones CHECK que generaban el conflicto y conservamos las claves primarias y foráneas. Esta decisión permitió ejecutar las tablas, aunque reconocemos que dejó de validarse automáticamente que un estudiante no pudiera seguirse a sí mismo o ser su propio mentor.

3. Dificultades con SQL Server y elección de SQLite

Inicialmente habíamos considerado SQL Server como tercer gestor. Sin embargo, durante el trabajo con el editor en línea encontramos dificultades para ejecutar algunas consultas del diccionario físico y visualizar sus resultados de manera organizada.

Por esta razón, decidimos utilizar SQLite como alternativa. Este gestor nos permitió implementar las 15 tablas sin necesidad de configurar un servidor independiente y obtener la información necesaria para documentar la estructura física de nuestra base de datos.

4. Manejo de datos semiestructurados

Otra decisión técnica estuvo relacionada con el almacenamiento de información JSON. Identificamos que PostgreSQL, MySQL y SQLite utilizan mecanismos diferentes para trabajar con este formato.

Por ello, adaptamos la definición del campo destinado a las preferencias de los estudiantes, teniendo en cuenta las características de cada gestor.

5. Generación y organización de los diccionarios físicos

También encontramos dificultades al consultar y exportar los diccionarios físicos, debido a que algunos editores mostraban resultados extensos o con formatos poco prácticos para su presentación.

Para solucionar esta situación, organizamos la información obtenida de cada gestor, procurando conservar los datos necesarios sobre tablas, columnas, tipos de datos, claves y restricciones.

Conclusión de las decisiones técnicas

Estas dificultades nos permitieron comprender que implementar una base de datos en diferentes gestores no consiste únicamente en copiar y pegar instrucciones SQL. Fue necesario interpretar los mensajes de error, reconocer las particularidades de cada motor y realizar ajustes técnicos.

Consideramos que este proceso fortaleció nuestros conocimientos sobre el diseño físico de bases de datos y la importancia de verificar que las estructuras implementadas correspondan al modelo relacional propuesto.

9. Ventajas, limitaciones y mejoras

Ventajas

Una de las principales ventajas de nuestro trabajo fue lograr implementar el modelo relacional de la Red Social Estudiantil Pascualina en tres gestores diferentes: PostgreSQL, MySQL y SQLite. Esto nos permitió comparar sus características y comprender cómo un mismo diseño puede adaptarse a distintas tecnologías.

También conseguimos organizar la información en 15 tablas relacionadas, establecer claves primarias y foráneas, e incorporar el manejo de datos semiestructurados mediante JSON.

Además, la elaboración de los diccionarios físicos nos permitió documentar la estructura de cada implementación y facilitar su revisión.

Limitaciones

Durante el desarrollo identificamos algunas limitaciones. En primer lugar, las diferencias de sintaxis y funcionamiento entre los gestores hicieron necesario modificar ciertas instrucciones SQL.

En MySQL tuvimos que retirar dos restricciones CHECK para resolver conflictos con las acciones referenciales, lo que dejó algunas reglas de validación pendientes de implementar mediante otro mecanismo.

Por otra parte, SQLite presenta limitaciones para aplicaciones que requieren numerosas escrituras simultáneas, por lo que sería necesario evaluar su conveniencia si Pascualina llegara a utilizarse como una red social con muchos usuarios activos.

Finalmente, nuestras pruebas se concentraron principalmente en la creación de las tablas y la revisión de sus estructuras, por lo que todavía sería necesario comprobar de manera más amplia el comportamiento de la base de datos con información real.

Propuestas de mejora

Como mejoras futuras, proponemos incorporar datos de prueba que permitan verificar las operaciones de inserción, actualización y eliminación, así como evaluar el funcionamiento de las relaciones entre las tablas.

También consideramos importante implementar mecanismos adicionales de validación en MySQL para impedir que un estudiante pueda seguirse a sí mismo o registrarse como su propio mentor.

Otra mejora sería analizar el rendimiento de las consultas más frecuentes, crear índices cuando sean necesarios y fortalecer los mecanismos de seguridad para proteger la información de los estudiantes.

Finalmente, si Pascualina llegara a desarrollarse como una aplicación real, sería conveniente evaluar cuál de los tres gestores ofrece mejores condiciones de rendimiento, concurrencia y mantenimiento según la cantidad de usuarios y las necesidades del sistema.

10. Reflexiones individuales

Jesus:

Durante el desarrollo de esta actividad comprendí que trabajar con bases de datos va mucho más allá de crear tablas y escribir código SQL. Al principio pensé que implementar nuestro modelo en tres gestores sería relativamente sencillo, pero me di cuenta de que cada uno tiene características y reglas diferentes.

Uno de los mayores retos fue enfrentar los errores que aparecieron durante la ejecución de los códigos, especialmente en MySQL. Hubo momentos en los que tuvimos que revisar varias veces las instrucciones para entender qué estaba fallando. Aunque fue un proceso algo frustrante, me ayudó a comprender la importancia de prestar atención a los

mensajes de error.

También aprendí que las claves primarias, las claves foráneas y las restricciones cumplen un papel fundamental para mantener la integridad de la información. Además, trabajar con PostgreSQL, MySQL y SQLite me permitió conocer distintas maneras de implementar un mismo modelo relacional.

Considero que esta actividad fortaleció mis conocimientos técnicos y me enseñó a ser más paciente al resolver problemas. Me llevo la experiencia de haber enfrentado dificultades reales y de comprender que equivocarse, investigar y corregir también forma parte del aprendizaje.

Jose

Esta actividad me permitió entender mejor cómo se lleva un modelo de base de datos desde su diseño hasta su implementación. En las tareas anteriores habíamos trabajado principalmente en las tablas y sus relaciones, pero en esta ocasión pude ver cómo esa estructura se convierte en instrucciones SQL que deben funcionar en diferentes gestores.

Algo que me llamó mucho la atención fue descubrir que, aunque PostgreSQL, MySQL y SQLite permiten trabajar con bases de datos relacionales, no utilizan exactamente la misma sintaxis. Esto me hizo comprender que es importante conocer las características de cada herramienta antes de comenzar a trabajar con ella.

También aprendí sobre la utilidad de los diccionarios físicos, porque al principio los veía simplemente como tablas llenas de información, pero después entendí que sirven para documentar y verificar cómo está construida una base de datos.

Otra parte importante de esta experiencia fue el trabajo en equipo. Tuvimos que compartir información, revisar errores y buscar soluciones para avanzar. No todo salió bien en el primer intento, pero logramos superar las dificultades y completar las implementaciones.

Personalmente, considero que esta tarea me ayudó a relacionar mejor la teoría con la práctica. Ahora tengo una idea más clara de cómo funcionan los gestores de bases de datos y de la importancia de diseñar estructuras organizadas, documentadas y coherentes con las necesidades de un proyecto.